

SALESTORM – REQUIREMENTS & ASSUMPTIONS DOCUMENT

1. SYSTEM OVERVIEW

SALESTORM is a high-scale e-commerce flash-sale platform designed to handle up to 10,000 concurrent customers competing for a limited inventory of 100 units. The system must prevent overselling, avoid duplicate reservations and payments, and maintain a reliable order lifecycle from product selection to final delivery.

The platform consists of product discovery, cart management, inventory reservation, checkout, payment, order processing, shipment, notification and delivery tracking. The architecture must remain scalable and reliable during normal traffic as well as sudden flash-sale traffic spikes.

2. FUNCTIONAL REQUIREMENTS

2.1 Customer Management

- Customers should be able to register, authenticate and manage their accounts.
- Customer information should be associated with carts, reservations and orders.
- Authentication and authorization should be applied before accessing protected operations.

2.2 Product and Cart Management

- Customers should be able to view products, categories, prices and product availability.
- Customers should be able to add products to their cart.
- Customers should be able to update quantities and remove products from the cart.
- Product information should be available through REST APIs.
- Frequently accessed product information may be served through caching to reduce database load.

2.3 Inventory and Reservation Management

- The system must maintain the available, reserved and sold quantity of each product.
- Inventory must be reserved before completing payment.

- A reservation should have an expiry time so that unpaid reservations do not block inventory permanently.

- Inventory should follow the lifecycle:

AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD

- If payment fails or a reservation expires, the reserved quantity must be released back to available inventory.

- Concurrent requests for the same product must be controlled so that the same unit cannot be reserved by multiple customers.

- Duplicate reservation requests must not create multiple reservations and should be handled using idempotency.

2.4 Checkout

- Checkout should validate the selected products and their reservations before payment.

- The system should calculate the final order amount based on product quantity and applicable pricing.

- Checkout should coordinate inventory reservation, order creation and payment initiation.

- Invalid, expired or unavailable reservations should prevent checkout from proceeding.

- Checkout requests should support idempotency to avoid duplicate transactions.

2.5 Payment Management

- The system should support successful payments, failed payments and payment timeouts.

- Each payment should have a unique payment reference and provider transaction reference.

- Duplicate payment requests must not create duplicate payment transactions.

- Payment processing should use an idempotency key.

- If payment fails, the associated reservation should be released.

- If payment times out, the system should not immediately assume failure because the external payment gateway may have completed the transaction.

- Timed-out payments should remain in a pending state until their status is confirmed through reconciliation.

- If payment succeeds while the Order Service is temporarily unavailable, the payment result must be stored and delivered through reliable asynchronous processing.

2.6 Order Management

- An order should be created and confirmed after successful payment processing.
- The order should maintain a clearly defined lifecycle:

CREATED → PAYMENT_PENDING → CONFIRMED → PROCESSING → SHIPPED →
OUT_FOR_DELIVERY → DELIVERED

- Orders should contain customer, product, quantity, price and payment-related information.
- Order operations should be idempotent so that duplicate requests or events do not create duplicate orders.
- Failed payments or expired reservations should result in appropriate order cancellation.
- The system should support recovery when dependent services such as payment or order services become temporarily unavailable.

2.7 Shipment and Delivery Management

- A shipment should be created for a successfully confirmed order.
- Shipment information should contain shipment status and tracking information.
- Customers should be able to track their shipment.
- Shipment updates should progress the order through shipping, out-for-delivery and delivered states.
- Delivery events should trigger appropriate customer notifications.

2.8 Notification Management

- The system should send notifications for important events such as successful orders, payment results, shipment updates and delivery completion.
- Notifications should be generated asynchronously where possible so that notification failures do not block core transactions.
- Failed notification events should be retried or moved to a dead-letter queue when necessary.

2.9 API Management

- REST APIs should be provided for products, cart, reservations, checkout, payments, orders and shipment tracking.
- APIs should support authentication and authorization.
- APIs should validate request data before processing.
- Appropriate HTTP status codes should be returned for success, validation errors, unauthorized requests, conflicts, rate limits and service failures.
- Idempotency should be supported for operations such as reservation, checkout, payment and order creation.

3. NON-FUNCTIONAL REQUIREMENTS

3.1 Scalability

- The system should support approximately 10,000 requests per second under normal high-load conditions.
- The architecture should be capable of handling extreme flash-sale traffic and reason about traffic levels up to approximately 500,000 requests per second.
- Services should support horizontal scaling so additional instances can be added during traffic spikes.
- Load balancing, caching and asynchronous processing should be used to distribute traffic efficiently.

3.2 Performance

- Product and read-heavy operations should have low response latency through caching and horizontal scaling.
- Inventory reservation should provide fast and deterministic processing even when many customers request the same product simultaneously.
- Requests that cannot be processed immediately should be controlled using rate limiting, queueing or rejection rather than allowing the system to become overloaded.

3.3 Consistency

- Inventory consistency is a strict requirement.
- Inventory must never become negative.
- The number of successful reservations and sales must never exceed the available stock.
- The last available unit must be deterministically reserved by only one successful request.
- PostgreSQL should act as the authoritative source for inventory and transactional data.
- Redis should be used only for caching and should not be treated as the source of truth for inventory.

3.4 Reliability

- The system should continue operating correctly during temporary service, database or payment gateway failures.
- Failed operations should use appropriate retry mechanisms.
- Important asynchronous events should be stored using a durable messaging mechanism.
- Dead-letter queues should be used for messages that repeatedly fail processing.
- Payment and order processing should support reconciliation to recover from uncertain states.

3.5 Availability

- Services should support horizontal scaling and graceful failure recovery.
- Failure of non-critical services such as notification should not prevent successful payment or order creation.
- Critical services should be designed to recover from temporary failures without losing important transactions or events.

3.6 Security

- Authentication and authorization should be enforced for protected APIs.
- All communication should use HTTPS.
- Rate limiting should protect APIs from excessive or abusive traffic.

- Input validation should prevent invalid or malicious requests.
- Sensitive payment information should be handled securely and should not be unnecessarily stored.
- Secrets and credentials should be managed using a secure secrets-management mechanism.
- Important security and business operations should be recorded in audit logs.

3.7 Observability

- The system should collect structured logs, application metrics and distributed traces.
- Important metrics should include request rate, response latency, error rate, reservation failures, payment failures, order conversion and queue backlog.
- Alerts should be generated for conditions such as inventory inconsistencies, high payment failure rates, service failures and growing message queues.
- Distributed tracing should allow a transaction to be followed across services from checkout through payment and order creation.

4. SYSTEM CONSTRAINTS

- The system must support approximately 10,000 concurrent customers.
- The available inventory for the flash-sale scenario is 100 units.
- Payment success rate is assumed to be 95%.
- Payment failure rate is assumed to be 5%.
- Duplicate requests may account for approximately 2% of requests.
- The Order Service may be unavailable for approximately 30 seconds during the failure scenario.
- Traffic may increase by up to 50× during a flash sale.
- Overselling is strictly not allowed.
- Duplicate payments and duplicate orders are strictly not allowed.
- External payment and delivery services may experience delays, failures or timeouts.

5. ASSUMPTIONS

- PostgreSQL is the authoritative transactional database for customers, inventory, reservations, orders and payments.
- Inventory reservation is performed using database transactions and concurrency control such as row-level locking or atomic updates.
- Redis is used for caching frequently accessed data and is not treated as the authoritative inventory store.
- Every inventory reservation has an expiry time.
- Idempotency keys are generated or supplied for operations where duplicate requests can cause financial or inventory inconsistencies.
- Payment timeout represents an unknown result rather than an immediate failure.
- A reconciliation process checks the payment provider to determine the final payment status.
- A durable message broker is available for important asynchronous events.
- Event consumers are idempotent because messages may be delivered more than once.
- Payment and delivery gateways are external dependencies and may fail or become temporarily unavailable.
- Temporary failures are handled through retries, backoff, reconciliation and compensation where required.
- Order and payment events are persisted reliably so that important transactions are not lost during service failures.

6. KEY GUARANTEES

- Successful inventory reservations can never exceed the available stock.
- Inventory quantity can never become negative.
- Only one customer can successfully reserve the final available unit.
- Duplicate reservation requests do not create duplicate reservations.
- Duplicate payment requests do not create duplicate payment transactions.
- Duplicate order requests or events do not create duplicate orders.
- Failed or expired reservations are released back to available inventory.

- Payment timeout is safely reconciled instead of being incorrectly treated as failure.
- A successful payment is not lost even if the Order Service is temporarily unavailable.
- Important events can be retried without creating duplicate business transactions.
- Successful purchases eventually reach a valid order state.
- The system can scale horizontally to handle increased traffic.